PROJECT REPORT ON

A COMPARATIVE STUDY OF GRAPH-BASED RETRIEVAL-AUGMENTED GENERATION FRAMEWORKS: PATHRAG, S-PATHRAG, AND LIGHTRAG

SUBMITTED TO

SVKM'S NMIMS (DEEMED-TO-BE UNIVERSITY)

IN PARTIAL FULFILLMENT OF THE DEGREE OF

MASTER OF SCIENCE

IN

DATA SCIENCE

BY

Rohini Pedamkar	**86092400109**
Batul Rampurawala	**86092400082**
Amandeep Singh Saini	**86092400100**
Omkar Mahajan	**86092400102**

April, 2026

NILKAMAL SCHOOL OF MATHEMATICS,
APPLIED STATISTICS & ANALYTICS

MUMBAI – 400056

PROJECT REPORT ON

A COMPARATIVE STUDY OF GRAPH-BASED RETRIEVAL-AUGMENTED GENERATION FRAMEWORKS: PATHRAG, S-PATHRAG, AND LIGHTRAG

SUBMITTED TO

SVKM'S NMIMS (DEEMED-TO-BE UNIVERSITY)

IN PARTIAL FULFILLMENT OF THE DEGREE OF

MASTER OF SCIENCE

IN

DATA SCIENCE

BY

Rohini Pedamkar	**86092400109**
Batul Rampurawala	**86092400082**
Amandeep Singh Saini	**86092400100**
Omkar Mahajan	**86092400102**

Dean

(Dr. Sushil Kulkarni)

April, 2026

NILKAMAL SCHOOL OF MATHEMATICS,

APPLIED STATISTICS & ANALYTICS

MUMBAI – 400056

CERTIFICATE

This is to certify that the work described in this thesis entitled **"A Comparative Study of Graph-Based Retrieval-Augmented Generation Frameworks: PathRAG, S-PathRAG, and LightRAG"** has been carried out by **Rohini Pedamkar (86092400109), Batul Rampurawala (86092400082), Amandeep Singh Saini (86092400100),** and **Omkar Mahajan (86092400102)** under our supervision. We certify that this is their bonafide work. The work described is original and has not been submitted for any degree to this or any other University.

Date:

Place:

Internal Mentor	**Dean**
(Prof. Pratik Desai)	(Dr. Sushil Kulkarni)
Date:	**Date:**

NILKAMAL SCHOOL OF MATHEMATICS,
APPLIED STATISTICS & ANALYTICS

NILKAMAL SCHOOL OF MATHEMATICS,

APPLIED STATISTICS & ANALYTICS

MUMBAI – 400056

Year: _____

Statement by the Candidate

This is to submit that this written submission in our report entitled **"A Comparative Study of Graph-Based Retrieval-Augmented Generation Frameworks: PathRAG, S-PathRAG, and LightRAG"** represents our ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the sources.

We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

We understand that any violation of the above will be cause for disciplinary action by the school and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Rohini Pedamkar	86092400109
Batul Rampurawala	86092400082
Amandeep Singh Saini	86092400100
Omkar Mahajan	86092400102

Forwarded through,

Prof. Pratik Desai
Nilkamal School of Mathematics, Applied Statistics & Analytics,
NMIMS University, V.L. Mehta Road,
Vile Parle (W), Mumbai–400056, India.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide, **Prof. Pratik Desai**, for the invaluable guidance, constant encouragement, and insightful feedback provided throughout the course of this project. We are equally grateful to **Dr. Sushil Kulkarni**, Dean of the Nilkamal School of Mathematics, Applied Statistics & Analytics, for his support and for fostering the academic environment in which this work was possible.

We are deeply thankful to the faculty of the *Nilkamal School of Mathematics, Applied Statistics & Analytics, NMIMS*, for providing the academic environment and resources necessary to carry out this research. We also acknowledge the authors of the PathRAG, S-PathRAG, and LightRAG frameworks, whose publications form the foundation of this study.

Finally, we wish to thank our families and fellow students for their continued support during our Master's programme.

Rohini Pedamkar	86092400109
Batul Rampurawala	86092400082
Amandeep Singh Saini	86092400100
Omkar Mahajan	86092400102

Table of Contents

Table of Contents i

List of Figures iv

List of Tables v

1 Abstract 1

2 Introduction 2

2.1 Background 2

2.2 Problem Statement 4

2.3 Objectives 4

2.4 Scope of the Study 4

2.5 Significant Contributions 4

3 Rationale 6

4 Literature Review 7

4.1 Large Language Models 7

4.1.1 Architecture and Training 7

4.1.2 Capabilities and Limitations 7

4.2 Naive Retrieval-Augmented Generation 8

4.2.1 The Three-Stage Pipeline 8

4.2.2 Strengths and Structural Limitations 8

4.3 Graph-Based Retrieval-Augmented Generation (GraphRAG) 8

4.3.1 Core Concept 8

4.3.2 The Five-Component Framework 9

4.3.3 How GraphRAG Is Built Through LLMs 9

4.4 Systematic Comparison: LLM, Naive RAG, GraphRAG 10

4.5 Open Research Challenges 10

4.6 Position of this Report 11

4.7 A Practical Observation 11

5 Research Envisaged, Objectives and Plan of Work 13

5.1 Status of the Problem 13

5.2 Hypotheses . . . 13
5.3 Research Objectives . . . 13
5.4 Plan of Work . . . 13

6 Data Preparation **14**
6.1 Corpus-to-Graph Construction . . . 14
6.2 Storage Layout . . . 14
6.3 Framework-Specific Preparation . . . 14
6.4 Choices in Embedding, Indexing, and Chunking . . . 15
6.4.1 Chunking Strategies . . . 15
6.4.2 Embedding Models . . . 15
6.4.3 Vector Index Choice . . . 16
6.4.4 Choices Made in This Work . . . 16

7 Methodology **17**
7.1 PathRAG: Path-Based Retrieval . . . 17
7.1.1 Node Retrieval . . . 17
7.1.2 Path Retrieval with Flow-Based Pruning . . . 17
7.1.3 Path Scoring and Answer Generation . . . 18
7.2 S-PathRAG: Semantic-Aware Iterative Retrieval . . . 18
7.2.1 Semantic-Aware Path Retrieval . . . 18
7.2.2 Hybrid Path Enumeration . . . 19
7.2.3 Differentiable Path Scoring and Verification . . . 19
7.2.4 Latent Path Injection . . . 19
7.2.5 Iterative Retrieval (Neural-Socratic Graph Dialogue) . . . 19
7.3 LightRAG: Dual-Level Lightweight Retrieval . . . 19
7.3.1 Graph-Based Text Indexing and Profiling . . . 19
7.3.2 Dual-Level Retrieval . . . 19
7.3.3 Incremental Updating . . . 20
7.3.4 Reported Efficiency on the Legal Dataset . . . 20
7.4 Software and Tools . . . 20
7.5 LightRAG-Style Implementation . . . 20
7.5.1 Implementation Corpus . . . 21
7.5.2 Offline Indexing Stage . . . 21
7.5.3 Online Query Stage . . . 22
7.5.4 Software Stack . . . 22
7.6 Evaluation Criteria . . . 23

7.7 Complexity 23

8 Results and Discussion **25**

8.1 Comparative Findings 25

8.1.1 Retrieval Paradigm 25

8.1.2 Redundancy and Token Usage 25

8.1.3 Multi-Hop Reasoning and Hallucination Control 25

8.1.4 Domain Specialisation 25

8.2 Summary Comparison 25

8.3 Discussion 25

8.3.1 Where Each Framework Fails 26

8.4 Observation: Effect of Extraction LLM on Graph Quality 27

8.5 Limitations 29

9 Summary and Conclusion **30**

9.1 Summary 30

9.2 Conclusions 30

9.3 Scope for Future Work 30

References **31**

References **31**

A Supplementary Material **33**

A.1 Extraction Prompt 33

A.2 QA Evaluation Questions 33

A.3 Representative Gold Triples 34

A.4 Implementation Commands 34

A.5 Neo4j Visualisation 35

List of Figures

2.1 Architectural progression from LLMs to next-generation graph-based RAG. Each paradigm emerges as the failure mode of the previous one is identified. Adapted from Han et al. [15]. 3

7.1 Unified four-stage view of the PathRAG, S-PathRAG, and LightRAG pipelines. All three share the top row (graph construction → indexing → retrieval → generation) but differ in the specific techniques used at each stage (orange boxes). The dashed red arrow indicates the LLM-as-graph-constructor feedback loop. 17

7.2 Neo4j Browser showing the Cypher query `MATCH p=()-[:DIRECTED]->() RETURN p LIMIT 25;` executed against the knowledge graph produced by the LightRAG-style pipeline, together with the resulting graph visualisation. 22

A.1 Neo4j Browser view of the extracted knowledge graph. The central node corresponds to Napoleon Bonaparte and is surrounded by satellite entities linked via `DIRECTED` relationships. 35

List of Tables

4.1 Naive RAG three-stage pipeline. 8
4.2 The five-component GraphRAG framework [15]. 9
4.3 Pipeline transformation from Naive RAG to GraphRAG. 10
4.4 Dimension-by-dimension comparison of the three paradigms. 11

6.1 Comparison of graph construction choices across the three frameworks. 15

7.1 Reported retrieval and update cost of community-based GraphRAG vs LightRAG on the Legal benchmark, after Figure 2 of Guo et al. [8]. $C_{\max}$ is the generator context window and T_{extract}, C_{extract} are the per-update extraction cost in tokens and API calls. 20
7.2 Schema of the knowledge graph produced by the LightRAG-style implementation on the Napoleon corpus, as reported by the Neo4j browser after ingestion. . 21

8.1 Qualitative comparison of PathRAG, S-PathRAG, and LightRAG. 26
8.2 Reported efficiency gains of LightRAG versus community-based GraphRAG [8]. 26
8.3 Extraction quality of five LLMs on the gold reference chunk. Strict F1 counts a prediction as correct only if the entire (subject, predicate, object) triple matches the gold after case/whitespace normalisation; lenient F1 counts entity pairs regardless of predicate phrasing. 27
8.4 Full-corpus graph topology by extraction LLM. “#Triples” is the total number of extracted triples over the 148-chunk corpus; “#Components” is the number of weakly-connected components in the resulting multigraph. 28
8.5 Pairwise entity Jaccard overlap between extraction LLMs on the full corpus. A value of 1.00 indicates identical entity sets; lower values indicate the models extracted different entities from the same text. 28

A.1 Representative subset (15 of 60) of the manually-annotated gold reference triples. 34

1 Abstract

Retrieval-Augmented Generation (RAG) has become a standard technique for grounding Large Language Models (LLMs) in external knowledge. However, conventional vector-based RAG often returns redundant or disconnected chunks, which introduces noise and inflates token consumption. Graph-based RAG frameworks address this by structuring knowledge as entities and relationships, but they differ substantially in how retrieval is performed.

This project presents a comparative study of three recent graph-based RAG frameworks: **PathRAG**, which prunes a knowledge graph into relational paths using flow-based importance propagation; **S-PathRAG**, which extends PathRAG with semantic-aware path scoring, latent path injection, and iterative retrieval; and **LightRAG**, a lightweight dual-level retrieval system combining local entity lookup with high-level thematic reasoning. The study analyses their indexing strategies, retrieval mechanisms, and answer-generation pipelines, and contrasts them on the axes of retrieval quality, efficiency, and scalability.

The findings indicate that path-centric retrieval markedly reduces redundancy, semantic-aware iteration improves multi-hop reasoning, and dual-level retrieval delivers substantial cost-efficiency with near-real-time performance. Alongside this comparison, a LightRAG-style pipeline was implemented and run under five different extraction LLMs on a cancer drug-target corpus; this incidental experiment surfaces substantial variation in graph quality and topology across models, suggesting that the choice of extraction LLM is an under-studied dimension of graph-based RAG. The report concludes that the choice of framework depends on the trade-off between reasoning depth, semantic alignment, and deployment cost.

Keywords: Retrieval-Augmented Generation; Knowledge Graphs; PathRAG; S-PathRAG; LightRAG; Large Language Models; Multi-Hop Question Answering.

2 Introduction

2.1 Background

Large Language Models (LLMs) have demonstrated strong capabilities in natural language understanding and generation [25], but their architecture exposes three persistent weaknesses. First, the model's knowledge is *parametric*: encoded in frozen weights and impossible to update without retraining. Second, this parametric memory produces *hallucinations*, confident but factually incorrect outputs [22]. Third, the model cannot cite sources, making its claims impossible to audit.

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. [18] as a mitigation: rather than relying on parametric memory, the LLM is conditioned on passages retrieved from an external corpus at inference time. In its *naive* form, RAG chunks documents, embeds them, and retrieves the top-K nearest chunks by cosine similarity [4, 6]. This works well for single-fact retrieval but fails on queries requiring *multi-hop relational reasoning*, because chunks are treated as disconnected islands of text [2, 15].

Graph-based Retrieval-Augmented Generation (GraphRAG) addresses this gap by organising knowledge as a graph of typed entity–relation triples rather than a flat vector store. Queries are answered by retrieving *subgraphs* whose relational structure supports multi-hop inference, explainable reasoning paths, and substantially reduced hallucination [15, 20, 21]. Microsoft's GraphRAG implementation [2] further applies community detection (Leiden clustering) to produce multi-level summaries of the corpus.

Beyond vanilla GraphRAG, a new generation of frameworks targets its two dominant failure modes, retrieval *redundancy* and computational cost: **PathRAG** [1], **S-PathRAG** [5], and **LightRAG** [8]. These are the focus of the present report.

The three frameworks occupy different points along the practical trade-off between *retrieval quality*, *semantic alignment*, and *computational efficiency*. PathRAG prioritises structural selectivity: it trims the knowledge graph to a small set of relational paths, reducing token cost but relying on graph topology rather than query semantics. S-PathRAG adds a semantic scoring layer and an iterative refinement loop, which improves multi-hop reasoning and hallucination resistance but introduces additional model calls and training overhead. LightRAG keeps retrieval cheap by combining a flat entity lookup with a community-level summary, trading depth of reasoning for ease of deployment and incremental update. Understanding where each framework sits on this triangle is the central question this report addresses.

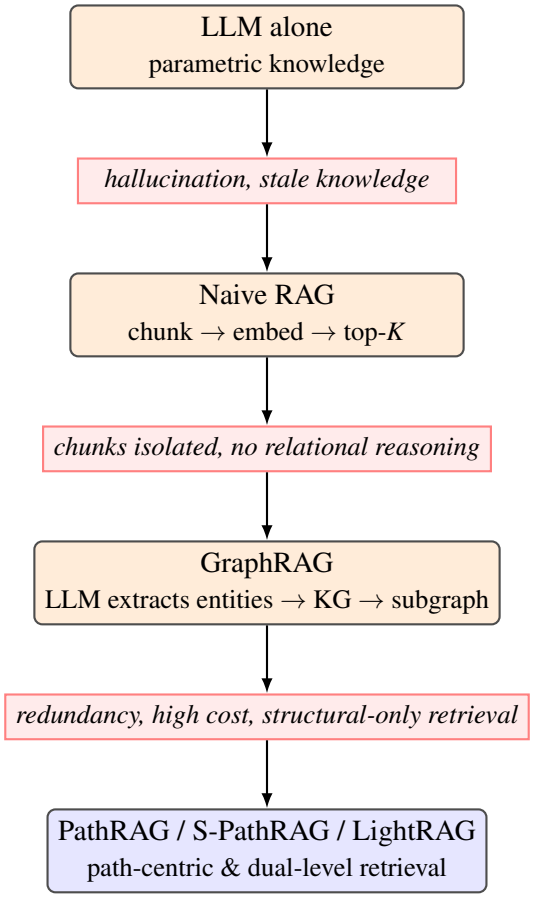

Figure 2.1: Architectural progression from LLMs to next-generation graph-based RAG. Each paradigm emerges as the failure mode of the previous one is identified. Adapted from Han et al. [15].

2.2 Problem Statement

Traditional graph-based RAG systems retrieve entire communities or local neighbourhoods around query-related nodes. This produces prompts that are rich in information but also saturated with irrelevant content, which degrades LLM reasoning quality and inflates token cost. The central problem, as highlighted in recent work [1] and in the project's reference video lecture [23], is not the *absence* of information but the *excess* of it. In parallel, purely structural retrieval methods fail to fully align retrieved paths with query semantics, and one-shot retrieval leaves no room for refinement when the initial evidence is insufficient.

This project therefore addresses the following research question:

> *How do recent graph-based RAG frameworks: PathRAG, S-PathRAG, and LightRAG differ in the way they index knowledge, retrieve evidence, and interact with the LLM, and what are the practical trade-offs between retrieval quality, semantic alignment, and computational efficiency?*

2.3 Objectives

1. To study the graph construction and indexing strategies used by PathRAG, S-PathRAG, and LightRAG.
2. To analyse the retrieval pipelines of each framework, focusing on path pruning, semantic scoring, and dual-level retrieval.
3. To compare the three frameworks on redundancy reduction, multi-hop reasoning support, and token efficiency.
4. To identify the conditions under which each framework is most suitable for real-world knowledge-intensive question answering.

2.4 Scope of the Study

The scope of this study is limited to graph-based RAG frameworks that operate on entity–relation knowledge graphs constructed from unstructured text. The work focuses on methodological comparison and does not attempt a full-scale empirical benchmark; where quantitative claims are made, they are taken from the original publications and clearly cited. The study does not cover purely vector-based RAG, agentic RAG, or multimodal extensions.

2.5 Significant Contributions

The primary contributions of this report are threefold. First, it offers a unified description of PathRAG, S-PathRAG, and LightRAG under a common four-stage pipeline of graph construction, indexing, retrieval, and generation, making it possible to talk about the three frameworks in a single vocabulary despite their differing design emphases. Second, it provides a structured comparison of the frameworks' strengths and limitations along the axes identified in Section 2.2,

including retrieval selectivity, semantic alignment, multi-hop reasoning capability, and computational profile. Third, it translates that comparison into practical guidance on framework selection, mapping application constraints such as deployment budget, update frequency, and reasoning depth onto the framework that best satisfies them.

In addition, a LightRAG-style implementation is exercised hands-on (Chapter 7). While implementing the pipeline, we noticed that the extraction LLM has a non-trivial effect on the resulting graph; a short side-observation of this effect is reported in Section 8.4.

3 Rationale

The rationale behind this study stems from the growing gap between the capability of LLMs and the practical cost of grounding them in large knowledge bases. Enterprise deployments increasingly require RAG systems that are simultaneously (i) accurate over multi-hop questions, (ii) efficient in token usage, and (iii) capable of continuous knowledge updates. A single framework that optimises one of these dimensions in isolation is of limited use; real-world systems must balance all three against a budget, and the balance point shifts as corpora, query patterns, and hardware change. This makes a direct, framework-by-framework comparison more valuable than yet another isolated architecture paper, because the comparative view exposes the trade-offs that inform deployment decisions.

Existing graph-based approaches each solve part of this problem but leave others open:

- GraphRAG-style community retrieval is thematically rich but computationally expensive.
- Naive vector RAG is cheap but structurally blind and redundancy-prone.
- Newer path-centric and dual-level methods promise both structure and efficiency, but practitioners lack a clear comparison.

A focused comparative study of PathRAG, S-PathRAG, and LightRAG is therefore timely and directly useful for researchers and engineers selecting a graph-based RAG architecture. The present work is further motivated by the reference video lecture on the evolution of graph-based RAG [23], which frames redundancy and hallucination as the two dominant failure modes that next-generation frameworks must address.

4 Literature Review

This chapter traces the architectural progression from plain Large Language Models, through Naive Retrieval-Augmented Generation, to Graph-based RAG. It situates the next-generation frameworks (PathRAG, S-PathRAG, LightRAG) examined in later chapters within this broader lineage. The treatment here draws heavily on Han et al. [15]'s holistic GraphRAG survey, Edge et al. [2]'s Microsoft GraphRAG implementation, and Erickson et al. [3]'s empirical work on knowledge-graph-validated LLM outputs.

4.1 Large Language Models

4.1.1 Architecture and Training

A Large Language Model is a neural network, typically based on the Transformer architecture, trained on massive corpora to model the conditional probability of the next token given all preceding tokens [25]. Through next-token prediction applied at enormous scale (billions of parameters, trillions of tokens), LLMs develop emergent capabilities including reasoning [24], summarisation, translation, and structured question answering.

The knowledge an LLM possesses is entirely *parametric*, encoded in frozen weights. At inference time, the model has no mechanism for consulting external sources or verifying its outputs.

4.1.2 Capabilities and Limitations

Three failure modes are central to motivating RAG [22]:

- **Hallucination:** confident but factually incorrect outputs. Hallucinations split into *intrinsic* (contradicting source material) and *extrinsic* (fabricated facts).
- **Knowledge cutoff:** parametric memory is frozen at the training date.
- **Lack of verifiability:** the model cannot cite the specific source from which a claim originates.

Why hallucination is structural, not incidental

LLMs do not retrieve facts, they interpolate. When asked a question, the model selects the most probable token sequence under its training distribution, not the most accurate. In domains where accurate answers are rare in training data, the distribution assigns high weight to plausible-sounding but incorrect outputs. RAG and GraphRAG address this by replacing interpolation with retrieval.

4.2 Naive Retrieval-Augmented Generation

4.2.1 The Three-Stage Pipeline

RAG was introduced by Lewis et al. [18] as a general framework for grounding LLM generation in retrieved evidence. In its naive form, the pipeline has three sequential stages [4, 6].

Table 4.1: Naive RAG three-stage pipeline.

Stage	What happens	Mechanism
1. Indexing	Documents are chunked and embedded into a vector space	Chunking + embedding model (BERT, text-embedding-ada)
2. Retrieval	Query is embedded; top-K nearest chunks returned	Cosine similarity / BM25 over a vector DB
3. Generation	Retrieved chunks concatenated into a prompt	LLM (GPT-4, Llama, etc.) generates answer

The retrieval step in stage 2 conventionally ranks candidate chunks by cosine similarity between the query embedding $\mathbf{e}_q$ and each chunk embedding $\mathbf{e}_d$:

$$\mathrm{sim}(q,d) \;=\; \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|\,\|\mathbf{e}_d\|}. \tag{4.1}$$

The top-K chunks under this score form the retrieved context passed to the generator [18].

4.2.2 Strengths and Structural Limitations

Naive RAG reduces hallucination for single-fact queries and bypasses the knowledge-cutoff problem. Between 2022 and 2024 it became the de facto standard for enterprise document Q&A. However, it has three structural limitations [15, 20]:

- **Multi-hop reasoning failure:** queries requiring facts from multiple documents connected by a relational path cannot be answered by top-K chunk retrieval.
- **Global understanding failure:** holistic questions about themes or trends cannot be answered by any finite passage set, because the answer is an aggregate property of the whole corpus.
- **No relational structure:** text chunks encode relations implicitly, but retrieval operates on surface similarity, not logical structure.

4.3 Graph-Based Retrieval-Augmented Generation (GraphRAG)

4.3.1 Core Concept

GraphRAG replaces the flat vector store with a *knowledge graph*: a structured database in which entities (nodes) are connected by typed relationships (edges). The system retrieves *subgraphs* rather than passages, enabling multi-hop traversal, explicit reasoning paths, and answers verifiable against a structured source [15, 21].

4.3.2 The Five-Component Framework

Han et al. [15] propose a unified framework in which every GraphRAG system decomposes into five components.

Table 4.2: The five-component GraphRAG framework [15].

Component	Function	Key techniques
Query Processor	Parse NL query into graph-compatible form	NER, relation extraction, query decomposition, query expansion
Graph Data Source	Store knowledge as nodes + typed edges	Explicit (citation links) or implicit (co-occurrence) construction; adjacency or edge lists
Retriever	Fetch relevant subgraphs / paths / nodes	Heuristic (BFS/DFS); learning-based (GNNs, embeddings); advanced (iterative, agentic)
Organiser	Post-process retrieved graph for generator	Graph pruning (semantic / syntactic / structural); reranking; verbalisation
Generator	Produce final answer from query + subgraph	LLM with verbalised graph; embedding-fusion; GNN-based

4.3.3 How GraphRAG Is Built Through LLMs

A critical and often underappreciated feature of modern GraphRAG is that the LLM is not merely the generator, it is also the *constructor* of the knowledge graph. Edge et al. [2] describe the Microsoft GraphRAG construction pipeline:

1. **Entity and relationship extraction:** the LLM reads each document chunk and identifies entities (with types and descriptions) and all pairwise relationships.
2. **Knowledge graph assembly:** extracted (entity, relation, entity) triples are assembled into an initial graph.
3. **Hierarchical clustering:** the Leiden algorithm identifies densely connected *communities*.
4. **Community summarisation:** the LLM generates natural-language summaries of each community.
5. **Query-time retrieval:** *local search* (entity traversal) for specific queries; *global search* (community summaries) for holistic queries.

This creates a feedback loop: the LLM's language understanding is used to build the very structure that grounds the LLM's future outputs.

The extraction step (1) is itself an autoregressive generation problem, in which the LLM, parameterised by θ, produces a target sequence y of triples conditioned on the input chunk x. The standard training objective is the token-level cross-entropy loss [18, 25]

$$\mathscr{L}_{\text{ext}}(\theta) \;=\; -\sum_{t=1}^{T} \log P_\theta(y_t \mid y_{<t}, x)\,, \tag{4.2}$$

which underpins both the entity/relation extractor and the downstream generator: at construction time the model emits triples, at query time the same model emits answers conditioned on the retrieved subgraph.

The hierarchical-clustering step that produces "communities" relies on the Leiden algorithm [14], which optimises a modularity objective Q over the partition $\{C_i\}$ of the graph G [2]:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j), \tag{4.3}$$

where A_{ij} is the weight of the edge between nodes i and j, k_i is the weighted degree of node i, m is the total edge weight, c_i is the community to which node i is assigned, and $\delta(\cdot,\cdot)$ is the Kronecker delta. Maximising Q produces densely-connected subgraphs whose summaries are then used as high-level retrieval units.

Empirical finding: the ChatBS-NexGen nutrition case study

Erickson et al. [3] submitted nutrition prompts to GPT-4o mini and Llama 3.1-8B for 100 patient records, validating each recommendation against the FoodKG knowledge graph. Key findings: (1) Both LLMs recommended high-glycaemic-index foods (GI $\geq$ 50) to diabetic patients, including honey (GI 60), maple syrup (GI 65), rice cake (GI 85), and watermelon (GI 75). (2) The global Jaccard similarity index for GPT-4o mini was only 0.05 across 10 repeated submissions for the same patient, revealing severe output instability. (3) FoodKG covered only 36.8% of distinct foods recommended (47.3% for diabetic-specific recommendations), illustrating the incompleteness of even well-curated knowledge graphs.

4.4 Systematic Comparison: LLM, Naive RAG, GraphRAG

Table 4.3: Pipeline transformation from Naive RAG to GraphRAG.

Stage	Naive RAG	GraphRAG	Key difference
Input	User query (text)	User query (text)	Same
Indexing	Chunk $\rightarrow$ embed $\rightarrow$ VDB	LLM extracts entities & relations $\rightarrow$ KG	Graph replaces vector store
Retrieval	Top-K cosine similarity	Entity linking $\rightarrow$ traversal / GNN	Relational paths vs. text similarity
Organise	Concatenate chunks	Prune, rerank, verbalise subgraph	Structure-aware filtering
Generate	LLM + retrieved text	LLM + structured subgraph	Grounded in verified graph facts
Output	Answer (text)	Answer + citable reasoning path	Explainability added

4.5 Open Research Challenges

Despite GraphRAG's advantages, several challenges remain unresolved [3, 15]:

Table 4.4: Dimension-by-dimension comparison of the three paradigms.

Dimension	LLM alone	Naive RAG	GraphRAG
Knowledge source	Parametric weights	Text chunks (vector DB)	Knowledge graph + text
Multi-hop reasoning	Limited / unreliable	Poor (chunks isolated)	Native – traverses paths
Hallucination risk	High	Moderate	Low (graph-grounded)
Explainability	Opaque	Citation-level	Full relational path
Dynamic update	Requires retraining	Re-index documents	Update graph incrementally
Consistency	Variable	Moderate	Structured – more stable
Compute cost	Low (inference only)	Moderate	Higher (graph build + query)
Best use case	General language	Simple factual QA	Complex multi-hop reasoning

- **Knowledge-graph completeness:** as in the FoodKG case, even curated graphs cover only a fraction of the entities mentioned in real queries.
- **Response consistency:** Jaccard-similarity scores as low as 0.05 across repeated identical prompts reveal that even graph-augmented pipelines are not reproducible without additional controls.
- **Query formulation:** translating natural language into graph-query form (Cypher, SPARQL, GQL) remains difficult, particularly for non-expert users.
- **Trustworthiness and safety:** relational structure can leak private information via neighbour traversal, raising new privacy and fairness concerns.

4.6 Position of this Report

The literature reviewed above establishes the *foundations* of graph-based RAG up to and including Microsoft-style community GraphRAG [2, 15]. The present report picks up where this literature ends: it examines three *next-generation* graph-based RAG frameworks PathRAG, S-PathRAG, and LightRAG, that explicitly target the computational-cost and redundancy limitations of community-based GraphRAG. The connection from this chapter to the rest of the report is visualised in Figure 2.1.

4.7 A Practical Observation

A secondary observation runs alongside the main comparative study. In all three frameworks examined in this report, the LLM plays a dual role: it *constructs* the knowledge graph via entity and relation extraction, and later *queries* the graph at answer-generation time. The original papers describe this dual role but treat the LLM component as interchangeable; none of them study how the specific LLM used for construction affects the quality of the resulting graph. While implementing a LightRAG-style pipeline for this study (Chapter 7), we observed that substituting different LLMs into the extraction step produces visibly different knowledge graphs

on identical input.

This is not a pathological edge case. Extraction LLMs make many small judgement calls per chunk: which spans are entities, which verbs count as relations, which entity types to assign, which redundant triples to collapse. Two models fed the same sentence can disagree on any of these and still produce internally-consistent graphs. Because all downstream retrieval and reasoning is conditioned on the extracted graph, any systematic bias introduced here propagates through the rest of the pipeline. Results of this small side-observation are reported in Section 8.4; the broader implication is that "LightRAG" is really a family of systems parameterised by the extraction backbone, not a single artefact.

5 Research Envisaged, Objectives and Plan of Work

5.1 Status of the Problem

Graph-based RAG is an active area of research. Three directions have emerged as particularly promising: path-centric retrieval (PathRAG), semantic-aware iterative retrieval (S-PathRAG), and lightweight dual-level retrieval (LightRAG). Each addresses different failure modes of earlier RAG systems, but no consolidated comparison exists that places them side by side in terms of methodology, cost, and reasoning capability.

5.2 Hypotheses

The project is guided by the following working hypotheses:

- **H1.** Path-centric retrieval (PathRAG) reduces prompt redundancy compared with neighbourhood- or community-based retrieval without sacrificing answer quality.
- **H2.** Introducing semantic-aware scoring and iterative refinement (S-PathRAG) improves multi-hop reasoning and reduces hallucinations relative to purely structural retrieval.
- **H3.** Dual-level retrieval (LightRAG) achieves substantially lower token and API cost than community-based GraphRAG while retaining retrieval accuracy.

5.3 Research Objectives

1. Characterise the indexing graph, retrieval pipeline, and generation strategy of each framework.
2. Compare the three frameworks along the axes of redundancy, semantic alignment, multi-hop reasoning, and efficiency.
3. Summarise the trade-offs and provide deployment-oriented recommendations.

5.4 Plan of Work

The project proceeded in four phases: (i) a *study phase* reviewing the original PathRAG, S-PathRAG, and LightRAG publications; (ii) an *analysis phase* decomposing each framework into a common four-stage pipeline (graph construction, indexing, retrieval, generation); (iii) a *comparison phase* performing cross-framework analysis using qualitative criteria and the quantitative claims reported in the source papers; and (iv) a *reporting phase* consolidating the findings, trade-off discussion, and recommendations.

6 Data Preparation

Since this report is a comparative methodological study, "data preparation" here refers to the process by which each framework converts a raw text corpus into a structured knowledge graph suitable for retrieval.

6.1 Corpus-to-Graph Construction

All three frameworks begin with an unstructured text corpus and use an LLM to extract structured knowledge. The shared steps are:

1. **Chunking:** source documents are split into semantically coherent chunks.
2. **Entity and relation extraction:** an LLM extracts Subject–Predicate–Object (SPO) triplets from each chunk [8].
3. **Node and edge creation:** entities become nodes and relationships become edges; each is annotated with a textual description.
4. **Embedding:** node, edge, and chunk embeddings are computed and stored for similarity search.

6.2 Storage Layout

LightRAG uses a three-tier storage strategy that is representative of the class as a whole [8]:

- **KV_STORAGE:** LLM response caches, text chunks, and document metadata.
- **VECTOR_STORAGE:** high-dimensional embeddings of entities, relations, and chunks.
- **GRAPH_STORAGE:** topology of the entity–relationship network (e.g., Neo4j or a local graph structure).

6.3 Framework-Specific Preparation

Although the offline construction phase looks superficially similar across the three frameworks, each attaches different auxiliary information to the extracted graph. The choice of auxiliary structure is what makes the downstream retrieval behaviour diverge.

- **PathRAG** pre-computes textual descriptions for each node and edge so that retrieved paths can be serialised into natural-language evidence [1].
- **S-PathRAG** augments the graph with learned embeddings produced by a contrastive path encoder, enabling semantic similarity scoring at retrieval time [5].
- **LightRAG** generates "profiles" for each node and edge as key–value pairs, where keys are optimised for rapid search and values summarise contextual role [8].

Table 6.1 summarises these differences across four design dimensions: the primary retrieval unit, the auxiliary profile attached to each node/edge, the focus of the index, and the supported

update mode. Notably, only LightRAG supports incremental updates out of the box; PathRAG and S-PathRAG both require re-indexing when the source corpus changes, which is a non-trivial cost for long-lived deployments.

Table 6.1: Comparison of graph construction choices across the three frameworks.

Aspect	PathRAG	S-PathRAG	LightRAG
Primary unit	Relational path	Semantic-aware path	Entity + dual-level index
Node/edge profile	Textual descriptions	Latent embeddings + text	Key–value profiles
Index focus	Structural importance	Semantic + structural	Local + global summaries
Update mode	Re-indexing	Re-indexing	Incremental

These construction-time choices largely predetermine what the retrieval step can and cannot do at query time. A graph whose edges carry only textual descriptions is well-suited to evidence verbalisation but cannot be scored by vector similarity; a graph with latent path embeddings supports semantic matching but needs a trained encoder; a graph with key–value profiles keeps both lookups and updates cheap but limits deep multi-hop traversal. The next chapter examines how each framework exploits its chosen graph representation during retrieval and generation.

6.4 Choices in Embedding, Indexing, and Chunking

Underneath every graph-based RAG framework sit three engineering choices that materially affect retrieval quality and cost: how the input text is split into chunks, what embedding model represents those chunks (and entities), and what data structure is used to search the resulting vectors at query time. The original PathRAG, S-PathRAG, and LightRAG papers describe these layers only briefly, but in practice they account for a non-trivial share of the cost-quality envelope.

6.4.1 Chunking Strategies

The simplest chunker splits text by a fixed token budget; richer variants respect sentence or paragraph boundaries (semantic chunking) or use a sliding window with overlap (recursive chunking). Smaller chunks improve recall by localising claims but lose surrounding context, hurting reasoning. Larger chunks preserve context but waste tokens on irrelevant material and increase prompt cost. Most production systems converge on a 300–800 token range with a small overlap. The pipeline implemented in this work uses 500-token fixed chunks with 50-token overlap, computed against the `cl100k_base` tokeniser used by GPT-4 and GPT-4o, so that token counts are consistent with the LLM's view of the input.

6.4.2 Embedding Models

The embedding model controls what counts as "similar" between a query and a chunk or entity. Two trade-offs dominate. First, larger embedding dimensionality (typically 768, 1024, 1536, or 3072) gives higher recall but increases storage and slows nearest-neighbour search. Second,

hosted embeddings (OpenAI's `text-embedding-3-small` and `text-embedding-3-large`; Cohere's `embed-v3.0` family and successors) are convenient but bind the system to an external API; open-source alternatives such as the BGE family [13] or Sentence-BERT-style encoders [12] can match hosted quality on many domains and run locally. For domain-specific corpora (biomedical, legal), specialised encoders typically outperform general-purpose ones at the cost of narrower applicability.

6.4.3 Vector Index Choice

Once chunks and entities are embedded, the retrieval step needs to find the top-K most similar vectors to a query embedding. Three families are common.

- **Brute-force** ($O(N)$ per query): linear scan over all N vectors. Simple and exact, suitable up to roughly 10^4 vectors.
- **IVF (Inverted File Index)**: cluster the vectors, then at query time scan only the nearest cluster(s). Sublinear on average; the canonical implementation is in Faiss [10].
- **HNSW (Hierarchical Navigable Small World)** [11]: a graph-based approximate nearest-neighbour index that delivers $O(\log N)$ query time at high recall, and is the de-facto choice for production RAG systems.

The trade-off is between index build time, query time, recall@K, and memory footprint. PathRAG and S-PathRAG, which retrieve over the knowledge graph rather than over raw chunks, treat the vector index as a means to identify anchor nodes; LightRAG treats it as the primary low-level retrieval mechanism, which is why its index cost appears explicitly in Table **??**.

6.4.4 Choices Made in This Work

To keep the LightRAG-style implementation in Section 7.5 reproducible and inspectable, each layer was instantiated minimally: 500-token `tiktoken` chunks with 50-token overlap; provider-native LLMs for entity and relation extraction (no separate embedding step in the construction pipeline, since the resulting graph is queried by graph traversal in Neo4j rather than by vector similarity); and a NetworkX in-memory multigraph for downstream metrics analysis. A production-grade deployment would replace the in-memory graph with a persistent graph-database backend and add an HNSW index over node descriptions to support the low-level retrieval branch of LightRAG's dual-level retriever.

7 Methodology

This chapter details the retrieval and generation methodologies of each of the three frameworks under study.

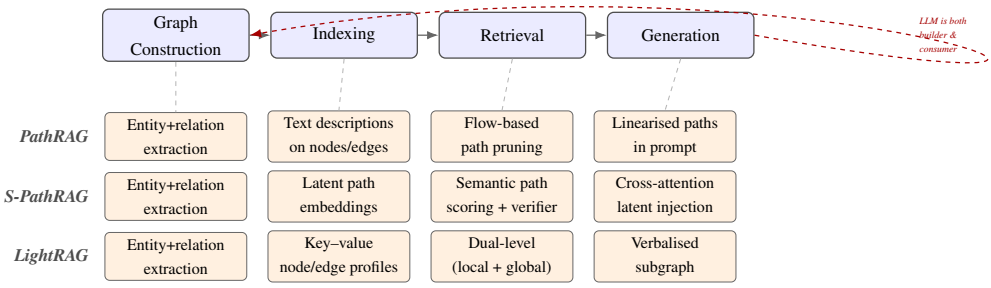

Figure 7.1: Unified four-stage view of the PathRAG, S-PathRAG, and LightRAG pipelines. All three share the top row (graph construction → indexing → retrieval → generation) but differ in the specific techniques used at each stage (orange boxes). The dashed red arrow indicates the LLM-as-graph-constructor feedback loop.

7.1 PathRAG: Path-Based Retrieval

PathRAG shifts retrieval from a *node-centric* paradigm to a *path-centric* paradigm [1]. Rather than returning all nodes or subgraphs related to a query, it retrieves a small number of structured relational paths between query-relevant entities.

7.1.1 Node Retrieval

The query is processed by an LLM to extract keywords, which are then embedded and compared against node embeddings using cosine similarity. The top-matching nodes form the *anchor set* for path search.

7.1.2 Path Retrieval with Flow-Based Pruning

Between anchor nodes, candidate paths are enumerated. To avoid combinatorial explosion, PathRAG applies a *flow-based pruning* mechanism governed by three principles:

- **Importance propagation:** relevance is propagated through the graph starting from query-related nodes.
- **Decay with distance:** nodes farther from the source receive lower importance.
- **Early pruning:** paths with low accumulated importance are discarded before full expansion.

Concretely, PathRAG defines a *resource flow* score $\mathscr{S}(v_i)$ over the knowledge graph $G = (V, E)$. The score at node v_i is the sum of decayed contributions from its predecessors, normalised

by each predecessor's out-degree [1]:

$$\mathscr{S}(v_i) \;=\; \sum_{v_j \in \mathscr{N}(\cdot, v_i)} \frac{\alpha \cdot \mathscr{S}(v_j)}{\left|\mathscr{N}(v_j, \cdot)\right|}, \tag{7.1}$$

where $\mathscr{N}(\cdot, v_i)$ is the set of predecessors of v_i (nodes whose flow enters v_i), $\mathscr{N}(v_j, \cdot)$ is the set of successors of v_j (used as the out-degree denominator that splits v_j's outgoing flow evenly), and $\alpha \in (0,1)$ is the global decay factor that shrinks the resource at every hop. Iterating Eq. (7.1) from query-anchored seed scores yields a stationary distribution in which nodes far from the anchors receive exponentially smaller flow, naturally realising the "decay with distance" principle without any explicit hop-distance term. Paths are then scored by aggregating $\mathscr{S}$ along them and pruned greedily once cumulative score falls below a threshold.

7.1.3 Path Scoring and Answer Generation

Each surviving path is assigned a reliability score based on node importance and the structural strength of the connection. The top-K paths are selected, linearised into natural-language evidence, and arranged strategically in the prompt to maximise LLM attention on high-value information.

7.2 S-PathRAG: Semantic-Aware Iterative Retrieval

S-PathRAG extends PathRAG with semantic awareness and an iterative refinement loop [5].

7.2.1 Semantic-Aware Path Retrieval

Paths are scored using both structural plausibility and semantic similarity to the query, so that retrieved evidence aligns with user intent rather than graph topology alone. S-PathRAG separates this into a per-edge weight and a path-level score. For an edge $e = (u, r, v)$ between nodes u and v via relation r, the edge weight combines a structural cost, a semantic dissimilarity term, and a relation prior [5]:

$$w_e \;=\; \alpha\, c_{\text{struct}}(e) \;+\; \beta\left(1 - \text{sim}(\ell_u, \ell_v)\right) \;+\; \gamma \pi_{\text{rel}}(r), \tag{7.2}$$

where $c_{\text{struct}}(e) \geq 0$ is a structural cost (e.g. inverse prior or length penalty), ℓ_u, ℓ_v are node embeddings, $\text{sim}(\cdot,\cdot)$ is cosine similarity (Eq. (4.1)), $\pi_{\text{rel}}(r)$ is an empirical or learnable relation prior, and $\alpha, \beta, \gamma \geq 0$ balance the three terms. A candidate path p for query q is then scored as the negative cumulative edge cost plus a learned path-level semantic match [5]:

$$\text{score}(p; q) \;=\; -\sum_{e \in p} w_e \;+\; \lambda_{\text{sem}}\, \text{sem}(p, q), \tag{7.3}$$

where $\text{sem}(p, q)$ is a learned semantic matcher (e.g. cosine similarity between a pooled path embedding and the query embedding) and $\lambda_{\text{sem}} \geq 0$ controls the relative weight of semantic

relevance against structural cost.

7.2.2 Hybrid Path Enumeration

Candidate paths are generated using a combination of shortest-path search, beam-based expansion, and random-walk exploration, balancing coverage against efficiency.

7.2.3 Differentiable Path Scoring and Verification

A learning-based scoring module contains:

- a *path scorer* that ranks paths;
- a *contrastive path encoder* that produces semantic representations;
- a *verifier* that filters out paths not supported by the knowledge graph.

The verifier is the key mechanism for reducing hallucinated or weakly grounded paths.

7.2.4 Latent Path Injection

Rather than serialising paths into long textual prompts, S-PathRAG encodes them into latent embeddings that are injected into the LLM via cross-attention. This preserves structured information while reducing token consumption.

7.2.5 Iterative Retrieval (Neural-Socratic Graph Dialogue)

The LLM emits diagnostic feedback indicating uncertainty or missing information. The framework uses this signal to update retrieval, producing a closed-loop refinement process in contrast to the one-shot retrieval of PathRAG.

7.3 LightRAG: Dual-Level Lightweight Retrieval

LightRAG is designed as a *lightweight* alternative to community-based GraphRAG [8], sitting between naive vector RAG and heavy graph traversal.

7.3.1 Graph-Based Text Indexing and Profiling

LightRAG uses an LLM to extract SPO triplets from the corpus and generates key–value profiles for every node and edge. Keys are optimised for rapid lookup; values summarise contextual role.

7.3.2 Dual-Level Retrieval

At query time, LightRAG performs retrieval at two granularities simultaneously:

- **Low-level retrieval:** targets specific entities and direct attributes, supporting precise "who/what/when" questions.
- **High-level retrieval:** targets global themes and cross-document concepts, supporting "why/how" reasoning.

A *hybrid (mix) mode* combines both levels to provide unified contextual grounding. Concretely, the low-level retriever ranks entity nodes by the cosine similarity (Eq. (4.1)) between their key

embedding and the query embedding, returning a top-k set that anchors the answer in specific facts. The high-level retriever performs the same similarity ranking over community-level summaries, returning a top-m set that gives the answer thematic context. In hybrid mode the two retrieved sets are merged before being passed to the generator, so the LLM sees both fine-grained evidence and high-level framing in a single prompt [8].

7.3.3 Incremental Updating

Unlike frameworks requiring full re-indexing on knowledge updates, LightRAG supports incremental insertion of new entities and relationships, making it suitable for evolving corpora.

7.3.4 Reported Efficiency on the Legal Dataset

The most concrete cost comparison in the LightRAG paper is on the Legal benchmark, where Guo et al. contrast token use and API calls against community-based GraphRAG [2, 8]. The reported figures are summarised in Table 7.1. At retrieval time the GraphRAG baseline issues approximately 610,000 tokens to the LLM versus fewer than 100 for LightRAG, and makes $\lceil 610{,}000/C_{\max} \rceil$ API calls (with $C_{\max}$ the generator's per-call context window) versus a single call. For incremental text updates the gap is even larger: GraphRAG must re-summarise $1{,}399 \times 2$ communities at 5,000 tokens each plus the entity-extraction cost T_{extract}, while LightRAG pays only T_{extract}. These numbers are the empirical basis for the headline "up to 90% fewer API calls and 6,000× fewer tokens" claim that the paper makes against GraphRAG.

Table 7.1: Reported retrieval and update cost of community-based GraphRAG vs LightRAG on the Legal benchmark, after Figure 2 of Guo et al. [8]. $C_{\max}$ is the generator context window and T_{extract}, C_{extract} are the per-update extraction cost in tokens and API calls.

	Retrieval		Incremental update	
Cost	**GraphRAG**	**LightRAG**	**GraphRAG**	**LightRAG**
Tokens	610,000	< 100	$1{,}399 \times 2 \times 5{,}000 + T_{\text{extract}}$	T_{extract}
API calls	$\lceil 610{,}000/C_{\max} \rceil$	1	$1{,}399 \times 2 + C_{\text{extract}}$	C_{extract}

7.4 Software and Tools

The methodological comparison draws on the reference implementations and descriptions provided by the original authors of PathRAG, S-PathRAG, and LightRAG. Alongside this comparison, a LightRAG-style pipeline was implemented to provide hands-on grounding (Section 7.5).

7.5 LightRAG-Style Implementation

To develop concrete familiarity with graph-based RAG, a LightRAG-style extraction-and-indexing pipeline was implemented end-to-end following the architectural outline of Guo et al. [8]. The implementation consists of two command-line entry points, `run_ingest.py` and

`run_query.py`, that together cover the pipeline's offline indexing stage and its online query stage.

7.5.1 Implementation Corpus

The implementation was exercised on an unstructured biographical corpus about **Napoleon Bonaparte**, drawn from Wikipedia. The corpus contains continuous prose describing Napoleon's campaigns, political relationships, and historical context, for example: *"Napoleon Bonaparte, the French military leader, launched a campaign in Italy in 1796 to defeat the Kingdom of Sardinia and the Austrian Empire."* This kind of text is well-suited to demonstrating graph-based RAG because it is densely entity-relational (persons, places, events, organisations) and supports multi-hop questions that naive vector RAG handles poorly.

7.5.2 Offline Indexing Stage

Indexing is triggered by:

```
python -m run_ingest.py
```

The script reads the raw text from `data/data.txt`, chunks it, and prompts an LLM to extract `(subject, predicate, object)` triples together with entity type labels and verbatim evidence spans. Extracted triples are written directly to a running Neo4j graph database so that the graph schema can be inspected interactively.

After ingestion completes, the Neo4j database reports the schema shown in Table 7.2: 178 nodes spanning 15 entity-type labels, connected by a single `DIRECTED` relationship type. Labels emerged directly from the LLM's extraction output, no manual schema was prescribed in advance, other than a few category hints in the system prompt.

Table 7.2: Schema of the knowledge graph produced by the LightRAG-style implementation on the Napoleon corpus, as reported by the Neo4j browser after ingestion.

Category	Value(s)
Node count	178
Node labels	artifact, base, category, concept, content, event, family, group, language, location, nation, organization, person, position, religion, UNKNOWN
Relationships	12 (single type: `DIRECTED`)
Property keys	`created_at`, `data`, `description`, `entity_id`, `entity_type`, `file_path`, `id`, `keywords`, `name`, `nodes`, `relationships`, `source_id`, `style`, `truncate`, `visualisation`, `weight`

The graph's central structural feature is a high-degree node corresponding to *Napoleon Bonaparte* himself, surrounded by satellite entities such as "Lucien Bonaparte", "Joseph Fouché", "Roger Ducos", "Paoli", "Michael von Melas" and "The Insurrection". Smaller chains link the central node to subsidiary concepts such as "The King's Swiss Guard" and "Toulon".

7.5.3 Online Query Stage

Query answering is triggered by passing a natural-language question directly to `run_query.py`:

```
python -m run_query.py "Summarize Napoleon's Italian campaign"
```

On this query, the pipeline combines a vector-similarity lookup over chunk embeddings with a graph traversal around query-relevant entity nodes. The retrieved evidence is verbalised and passed back to the LLM, which produces the final answer, in this case a concise summary that grounds its claims in the retrieved graph subset, for example: *"Napoleon Bonaparte, the French military leader, launched a campaign in Italy in 1796 to defeat the Kingdom of Sardinia and the Austrian Empire. His goal was to secure a strategic foothold in the region and weaken his enemies…"*

The workflow confirms the LightRAG value proposition described in the original paper: the same unstructured corpus supports both targeted single-fact retrieval ("who commanded the King's Swiss Guard?") and holistic summarisation ("Napoleon's Italian campaign") without requiring separate retrievers, because the graph structure itself encodes the relational context that both types of queries need.

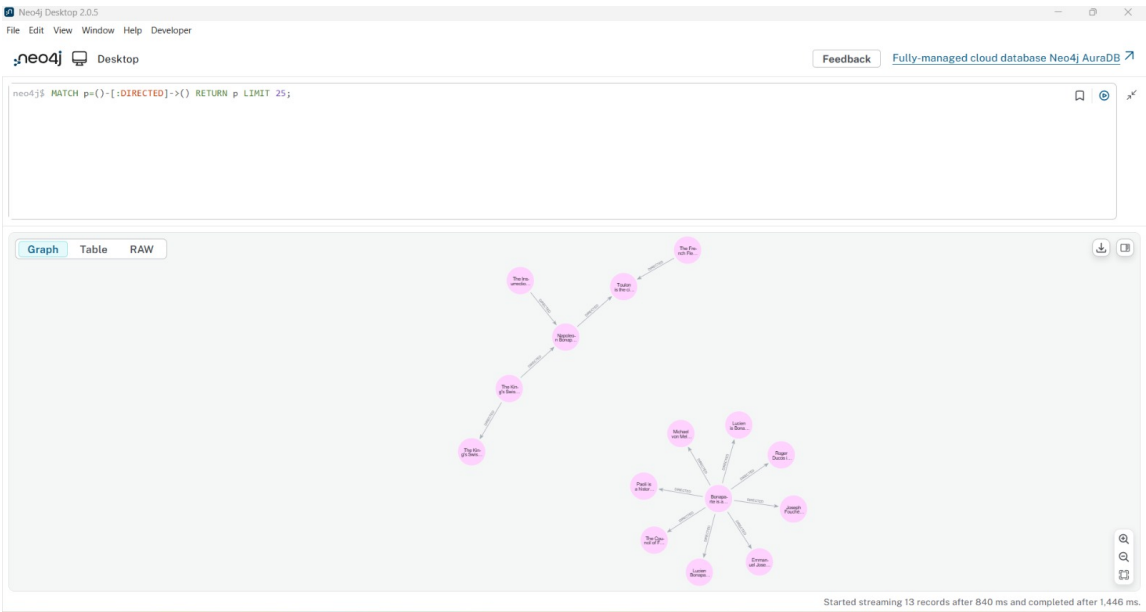

Figure 7.2: Neo4j Browser showing the Cypher query `MATCH p=()-[:DIRECTED]->() RETURN p LIMIT 25;` executed against the knowledge graph produced by the LightRAG-style pipeline, together with the resulting graph visualisation.

7.5.4 Software Stack

- Python 3.12 as the base language.
- Provider-native SDKs (OpenAI Python SDK; Anthropic Python SDK; Groq via OpenAI-compatible API) for LLM-driven extraction.
- `tiktoken` for tokenisation-aware corpus chunking.

- Neo4j Desktop (community edition) as the graph backend for the implementation demonstration.
- NetworkX [9] for downstream graph analysis and GraphML export.
- Pandas and scikit-learn for metrics; Matplotlib for plotting.

A second, smaller experiment, a cross-LLM comparison of extraction quality, was run on a separate **cancer drug-target** corpus using the same pipeline. That experiment is reported as an observation in Section 8.4.

7.6 Evaluation Criteria

The three frameworks are compared on five dimensions chosen to expose the trade-offs introduced in Section 2.2:

- **Retrieval quality:** precision and coverage of retrieved evidence.
- **Redundancy:** token overlap and noise in the prompt.
- **Multi-hop reasoning:** ability to chain evidence across multiple relations.
- **Hallucination rate:** frequency of unsupported answers.
- **Efficiency:** API calls, token consumption, and end-to-end latency.

These criteria are deliberately broader than a single numeric score: no framework dominates on all five simultaneously, so the comparison shown in Figure 7.1 and discussed in Chapter 8 is best read as a map of strengths rather than a leaderboard. Efficiency and retrieval quality, in particular, are often in direct tension: aggressive pruning reduces redundancy and token cost but risks discarding evidence that would have improved recall.

7.7 Complexity

Among the four frameworks reviewed, only S-PathRAG provides an explicit asymptotic cost analysis. Its authors state the dominant per-round complexity as [5]

$$C_{\text{round}} \;=\; O\big(L_{\text{GNN}} \cdot |E| \cdot d\big) \;+\; O\big(K \cdot \bar{\ell} \cdot d\big) \;+\; O\big(S \cdot M \cdot d\big), \tag{7.4}$$

where L_{GNN} is the number of GNN layers, $|E|$ the number of edges in the active subgraph, d the embedding dimensionality, K the number of candidate paths (capped at 200 in the reported experiments), $\bar{\ell}$ the average path length (≈ 2.8 in their runs), S the output token length, and M the number of injected key vectors used in cross-attention. S-PathRAG's empirical measurements report runtime scaling close to $T \propto |E|^{0.35}$ on knowledge graphs ranging from 1M to 100M edges, with peak GPU memory bounded below 12.8 GB.

The remaining frameworks publish qualitative scaling arguments but no closed-form asymptotic bound. PathRAG [1] relies on its flow-based pruning to keep the working set small and reports an empirical reduction in token consumption rather than a formal complexity. LightRAG [8] achieves its efficiency gains by replacing global community summarisation with key-value

profiles per node and a dual-level retrieval; its advantage is benchmarked against GraphRAG, not stated as Big-O. GraphRAG [2] couples a Leiden community-detection pass [14] with community-level summarisation, where Leiden is reported as “faster than Louvain” but without a formal bound. Standard nearest-neighbour search via HNSW is logarithmic in the corpus size [11], which underpins the sub-linear retrieval claims of LightRAG and Naive RAG. A direct, framework-agnostic complexity comparison is therefore left to future work.

8 Results and Discussion

8.1 Comparative Findings

8.1.1 Retrieval Paradigm

PathRAG is the first of the three to formalise *path-centric* retrieval. By propagating importance scores and pruning early, it consistently produces shorter, denser evidence than community- or neighbourhood-based retrieval [1]. S-PathRAG retains the path-centric view but enriches it with semantic alignment, which is critical for queries whose surface form does not match graph topology [5]. LightRAG takes a different approach: rather than choosing a single retrieval granularity, it performs retrieval at two levels simultaneously [8].

8.1.2 Redundancy and Token Usage

All three frameworks aim to reduce the prompt bloat associated with earlier graph-based RAG. PathRAG's flow-based pruning removes irrelevant branches before serialisation. S-PathRAG goes further by replacing long textual paths with latent embeddings injected via cross-attention, cutting token use while preserving structure. LightRAG reports the largest absolute gains against GraphRAG: up to a 90% reduction in API calls and up to 6000$\times$ reduction in token consumption for specific datasets, while maintaining or exceeding retrieval accuracy [8].

8.1.3 Multi-Hop Reasoning and Hallucination Control

PathRAG supports multi-hop reasoning implicitly through the structure of retrieved paths, but has no explicit hallucination filter. S-PathRAG introduces a verifier module that rejects plausible-looking but unsupported paths, and an iterative loop ("Neural-Socratic Graph Dialogue") that refines retrieval based on LLM feedback [5]. LightRAG handles multi-hop questions through its hybrid mode, where high-level themes are combined with low-level entity facts to form a more complete context.

8.1.4 Domain Specialisation

LightRAG has seen rapid adoption in specialised domains. In geosciences, a LightRAG-based GeoGPT pipeline achieved an F1-score of 0.835 for entity extraction, surpassing general-purpose models by 53–78% in remote-sensing benchmarks [7]. In clinical settings, Co-MedGraphRAG uses LightRAG's dual-level retrieval to generate personalised, evidence-based responses at low latency [19].

8.2 Summary Comparison

8.3 Discussion

The three frameworks are best viewed as complementary rather than strictly competing:

Table 8.1: Qualitative comparison of PathRAG, S-PathRAG, and LightRAG.

Criterion	PathRAG	S-PathRAG	LightRAG
Retrieval unit	Relational path	Semantic-aware path	Dual-level (entity + theme)
Pruning strategy	Flow-based importance	Learned scoring + verifier	Profile-guided lookup
Semantic alignment	Structural only	Structural + semantic	Embedding-based
Iterative refinement	No (one-shot)	Yes (Socratic loop)	No (one-shot)
Hallucination control	None explicit	Verifier module	Implicit (dual-grounding)
Token efficiency	High	Very high (latent inj.)	Very high (6000×)
Update mode	Re-indexing	Re-indexing	Incremental
Best suited for	Short multi-hop QA	Complex multi-hop QA	Real-time, large-scale QA

Table 8.2: Reported efficiency gains of LightRAG versus community-based GraphRAG [8].

Metric	LightRAG vs. GraphRAG
API calls (per query)	up to 90% fewer
Token consumption	up to 6000× lower
Retrieval accuracy	matched or exceeded

- **PathRAG** is a strong baseline when the priority is clean, structured evidence and the corpus is moderate in size.
- **S-PathRAG** is preferable when queries are semantically rich, multi-hop, or prone to hallucination, and when the additional cost of training the scorer/verifier is acceptable.
- **LightRAG** is the most attractive choice when deployment constraints (API cost, latency, incremental updates) dominate the design space.

A consistent theme across all three is the rejection of "more is better" retrieval. The contribution of recent graph-based RAG research is to demonstrate that *selecting less, more carefully* yields better LLM reasoning.

8.3.1 Where Each Framework Fails

A balanced reading also requires attention to the failure modes of each design.

- **PathRAG** is structurally driven, scoring paths by a flow-based propagation over the graph. It therefore struggles when the question–graph alignment is loose: a query whose key relations are stated metaphorically, colloquially, or across paraphrased entity names (e.g. "the drug Eli Lilly developed" versus a graph edge labelled `produced_by`) may receive low path-importance scores even when the relevant subgraph is present. Because retrieval is one-shot [1], the system has no mechanism to recover from such mismatches at query time.
- **S-PathRAG**'s differentiable path scorer and contrastive path encoder must be *trained* before they can be used [5]. This is a cold-start barrier: domains without a labelled path-relevance

dataset (most enterprise corpora, for instance) cannot benefit from the semantic-alignment gains until significant supervision is collected. The verifier inference at every retrieval step also adds per-query latency that may be unacceptable in interactive deployments.

- **LightRAG** keeps retrieval cheap by limiting itself to two granularities (low-level entity lookup and high-level community summaries) [8]. This is well-matched to questions that resolve at either of those scales, but is poorly matched to deep multi-hop chains in which the answer requires composing four or more relations that are not summarised in any single community. Such queries can return either irrelevant high-level themes or disconnected low-level facts, with no obvious mechanism in the framework to stitch them together.

8.4 Observation: Effect of Extraction LLM on Graph Quality

After the LightRAG-style pipeline in Section 7.5 was working on the Napoleon corpus, we repurposed it to run a small additional experiment on a separate **cancer drug-target** corpus ten Wikipedia articles spanning diseases, drugs, genes, and pathways, chunked into 148 passages of approximately 500 tokens each. A 60-triple gold reference was hand-annotated from one chunk (the "Inhibitors" section of the EZH2 article, the source of the canonical *tazemetostat* $\rightarrow$ *epithelioid sarcoma* multi-hop example in the graph-RAG literature). We then swapped five different LLMs into the extraction step and compared the resulting graphs against the gold. Precision, recall, and F1 in Table 8.3 are reported on the gold chunk only, so that each model is evaluated against identically-sized ground truth. Let Pred be the set of (subject, predicate, object) triples produced by an LLM and Gold be the manually-annotated reference set; standard information-retrieval definitions apply:

$$P = \frac{|\text{Pred} \cap \text{Gold}|}{|\text{Pred}|}, \quad R = \frac{|\text{Pred} \cap \text{Gold}|}{|\text{Gold}|}, \quad F_1 = \frac{2PR}{P+R}. \tag{8.1}$$

The strict variant computes Eq. (8.1) over full triples; the lenient variant computes it over (subject, object) entity pairs only, which absorbs paraphrase in the predicate [18].

Table 8.3: Extraction quality of five LLMs on the gold reference chunk. Strict F1 counts a prediction as correct only if the entire (subject, predicate, object) triple matches the gold after case/whitespace normalisation; lenient F1 counts entity pairs regardless of predicate phrasing.

LLM	Provider	Precision	Recall	Strict F1	Lenient F1
Claude Sonnet 4.6	Anthropic	0.58	0.30	**0.40**	0.48
Claude Opus 4.6	Anthropic	0.45	0.32	0.37	0.45
GPT-5 mini	OpenAI	0.29	0.20	0.24	0.27
Llama 3.3 70B	Groq	0.57	0.13	0.22	0.28
Llama 3.1 8B	Groq	0.05	0.02	0.03	0.08

Beyond per-triple quality, the topology of the resulting graph itself varies widely across models. Table 8.4 records the total number of extracted triples alongside the node count, edge count, and number of weakly-connected components for each LLM's graph built over the same

148-chunk corpus.

Table 8.4: Full-corpus graph topology by extraction LLM. "#Triples" is the total number of extracted triples over the 148-chunk corpus; "#Components" is the number of weakly-connected components in the resulting multigraph.

LLM	#Triples	#Nodes	#Edges	#Components
Claude Sonnet 4.6	2,926	2,130	2,926	85
Claude Opus 4.6	3,155	2,348	3,103	105
GPT-5 mini	4,305	4,315	4,305	387
Llama 3.3 70B	1,730	1,513	1,730	268
Llama 3.1 8B	2,272	1,961	2,272	273

Even when two models extract a similar number of triples, the entities they name can differ substantially. Table 8.5 quantifies this with a pairwise Jaccard index over the set of entity names produced by each model. For two extracted entity sets A and B, the Jaccard similarity is

$$J(A,B) \;=\; \frac{|A \cap B|}{|A \cup B|}, \tag{8.2}$$

with $J = 1$ indicating identical entity sets and $J = 0$ indicating disjoint extractions [3].

Table 8.5: Pairwise entity Jaccard overlap between extraction LLMs on the full corpus. A value of 1.00 indicates identical entity sets; lower values indicate the models extracted different entities from the same text.

	C. Sonnet	C. Opus	GPT-5 mini	L. 70B	L. 8B
Claude Sonnet 4.6	1.00	0.25	0.15	0.36	0.23
Claude Opus 4.6	0.25	1.00	0.02	0.21	0.14
GPT-5 mini	0.15	0.02	1.00	0.17	0.16
Llama 3.3 70B	0.36	0.21	0.17	1.00	0.30
Llama 3.1 8B	0.23	0.14	0.16	0.30	1.00

Three patterns stand out. First, the **Claude family dominates extraction quality**, Claude Sonnet 4.6 achieves the highest strict F1 (0.40) and Claude Opus 4.6 the highest recall (0.32), with both substantially ahead of the Groq-hosted Llamas on the same corpus. Second, **graph topology varies by an order of magnitude across LLMs** given the same input: GPT-5 mini produces roughly 2.5× as many triples as Llama 3.3 70B, suggesting very different notions of "what counts as a relation". Third, **cross-LLM entity overlap is low**: even the most-similar pair (Claude Sonnet and Llama 3.3 70B) share only 36% of extracted entities, and the least-similar pair (GPT-5 mini and Claude Opus) share just 2%. The same text produces visibly different knowledge graphs depending on which model reads it.

These results are an incidental observation from the implementation work, not a fully-controlled experiment. In particular, the strict-match metric is unforgiving of paraphrase (e.g. *"inhibits EZH2"* versus *"suppresses EZH2 activity"* score as a miss), which is one reason

lenient F1 runs roughly 1.5–2$\times$ higher than strict F1. The finding of substantive inter-LLM variation is nonetheless consistent across all three views: quality, topology, and entity overlap. A more rigorous evaluation, larger gold reference, multiple prompt variants, semantic-similarity matching, is left for future work (Section 9.3).

8.5 Limitations

- PathRAG is structure-driven, and some selected paths may not align with query meaning. Retrieval is one-shot and there is no mechanism to filter hallucinated paths [1].
- S-PathRAG introduces learning components (contrastive encoder, verifier) that require training data and compute, which may not be available in all deployments.
- LightRAG's dual-level retrieval is efficient but less expressive for deep chains of reasoning compared with dedicated path-based methods.
- The empirical observation in Section 8.4 is based on a single 60-triple gold chunk, one extraction prompt variant, and a small set of five LLMs. It is directional, not definitive.
- The 60-triple gold reference was annotated by one author and cross-reviewed by a second team member; disagreements were resolved by discussion. A formal Cohen's κ was not computed given the limited size of the reference set, but no triple in the final list was contested at review time.
- Treating each gold triple as an independent Bernoulli trial and using the standard binomial-proportion standard error $\sqrt{p(1-p)/n}$, we computed pairwise differences in precision and recall between the five models. The three conclusions that survive a $|\Delta|/\mathrm{SE}_\Delta > 2$ threshold are: (i) Llama 3.1 8B is reliably worse than every other model on both metrics, with effect sizes from 2.5σ (vs Llama 3.3 70B on recall) up to 5.3σ (vs Claude Sonnet on precision); (ii) Claude Sonnet has significantly higher precision than GPT-5 mini ($\Delta P = +0.29 \pm 0.11$, 2.5σ); and (iii) Claude Sonnet has significantly higher recall than Llama 3.3 70B ($\Delta R = +0.17 \pm 0.07$, 2.3σ). All other pairwise differences (Sonnet vs Opus on either metric, Opus vs GPT-5 mini, GPT-5 mini vs Llama 3.3 70B) fall below 2σ and cannot be distinguished from sampling noise on this dataset.
- Two notable findings emerge from those numbers. First, Claude Sonnet and Llama 3.3 70B have *statistically indistinguishable precision* (0.58 vs 0.57, 0.06σ): the Claude family's strict-F1 advantage comes almost entirely from higher recall, not from being "more careful" than the open-source large model. Second, GPT-5 mini and Llama 3.3 70B have nearly identical F1 (0.24 vs 0.22) but achieve it through opposite trade-offs: Llama 70B is high-precision low-recall while GPT-5 mini is the reverse. Strict F1 alone hides this qualitative difference.

9 Summary and Conclusion

9.1 Summary

This report has presented a comparative study of three recent graph-based Retrieval-Augmented Generation frameworks: PathRAG, S-PathRAG, and LightRAG. Each was decomposed into a common pipeline of graph construction, indexing, retrieval, and generation, and their mechanisms were contrasted in terms of retrieval paradigm, redundancy reduction, semantic alignment, multi-hop reasoning, and efficiency.

9.2 Conclusions

- Path-centric retrieval, as introduced by **PathRAG**, is an effective strategy for eliminating redundant information from graph-based RAG prompts.
- **S-PathRAG** advances the state of the art by integrating semantic-aware scoring, latent path injection, and an iterative Neural-Socratic retrieval loop, which together address the hallucination and one-shot-retrieval limitations of PathRAG.
- **LightRAG** achieves the strongest efficiency profile through dual-level retrieval and incremental updating, with reported reductions of up to 90% in API calls and up to 6000$\times$ in token consumption relative to community-based GraphRAG.
- The three frameworks are complementary: framework selection should be driven by the application's priorities across reasoning depth, semantic complexity, and deployment cost.

9.3 Scope for Future Work

- **Hybrid architectures:** combining LightRAG's dual-level retrieval with S-PathRAG's semantic verifier to obtain both efficiency and reliability.
- **Incremental real-time updates:** extending path-based methods to evolving knowledge graphs without full re-indexing.
- **Multimodal RAG:** generalising these frameworks to unified graphs containing text, tabular, and visual entities.
- **On-device inference:** leveraging the lightweight nature of LightRAG for edge deployments.
- **Empirical benchmarking:** a direct head-to-head evaluation of PathRAG, S-PathRAG, and LightRAG on standard multi-hop QA benchmarks.
- **LLM-substitution effects on graph quality:** following on from the observation in Section 8.4, a fuller study would expand the gold reference beyond one chunk, add semantic similarity-based matching to handle paraphrase, sweep prompt variants, and include closed-source flagships such as GPT-5 and Gemini Pro at full corpus scale.

References

[1] Chen, B., Guo, Z., Yang, Z., Chen, Y., Chen, J., Liu, Z., Shi, C., Yang, C., 2025. PathRAG: pruning graph-based retrieval augmented generation with relational paths. arXiv:2502.14902.

[2] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J., 2024. From local to global: a GraphRAG approach to query-focused summarization. arXiv:2404.16130. https://arxiv.org/abs/2404.16130.

[3] Erickson, J.S., Santos, H., Pinheiro, V., McCusker, J.P., McGuinness, D.L., 2025. LLM experimentation through knowledge graphs: towards improved management, repeatability, and verification. Web Semant. Sci. Serv. Agents World Wide Web 85, 100853.

[4] Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., Li, Q., 2024. A survey on RAG meeting LLMs: towards retrieval-augmented large language models, in: Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Min., pp. 6491–6501.

[5] Fu, R., Wang, Y., Xu, T., Liu, Y., Tang, W., Wu, W., Ma, X., Fong, S., 2026. S-Path-RAG: semantic-aware shortest-path retrieval augmented generation for multi-hop knowledge graph question answering. arXiv:2603.23512. https://arxiv.org/abs/2603.23512.

[6] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., 2023. Retrieval-augmented generation for large language models: a survey. arXiv:2312.10997.

[7] GeoGPT Consortium, 2026. Toward knowledge-enhanced geohazard intelligence: fusing geoscience LLMs and LightRAG. ResearchGate. https://bit.ly/GeoGPT-LightRAG.

[8] Guo, Z., Xia, L., Yu, Y., Ao, T., Huang, C., 2025. LightRAG: simple and fast retrieval-augmented generation. arXiv:2410.05779. https://arxiv.org/abs/2410.05779.

[9] Hagberg, A., Schult, D., Swart, P., 2008. Exploring network structure, dynamics, and function using NetworkX, in: Proc. 7th Python Sci. Conf. (SciPy), pp. 11–15.

[10] Johnson, J., Douze, M., Jégou, H., 2017. Billion-scale similarity search with GPUs. arXiv:1702.08734. https://arxiv.org/abs/1702.08734.

[11] Malkov, Y.A., Yashunin, D.A., 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. arXiv:1603.09320. https://arxiv.org/abs/1603.09320.

[12] Reimers, N., Gurevych, I., 2019. Sentence-BERT: sentence embeddings using Siamese BERT-networks, in: Proc. 2019 Conf. on Empirical Methods in NLP (EMNLP-IJCNLP), pp. 3982–3992. arXiv:1908.10084.

[13] Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., 2023. C-Pack: packaged resources to advance general Chinese embedding. arXiv:2309.07597. https://arxiv.org/abs/2309.07597.

[14] Traag, V.A., Waltman, L., van Eck, N.J., 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Sci. Rep.* 9, 5233.

[15] Han, H., Wang, Y., Shomer, H., Guo, K., Ding, J., Lei, Y., Halappanavar, M., Rossi, R.A., Mukherjee, S., Tang, X., He, Q., Hua, Z., Long, B., Zhao, T., Shah, N., Javari, A., Xia, Y., Tang, J., 2025. Retrieval-augmented generation with graphs (GraphRAG). arXiv:2501.00309.

[16] HKUDS, 2025. LightRAG GitHub repository: open-source implementation. https://github.com/HKUDS/LightRAG (accessed 14 April 2026).

[17] Kau, A., He, X., Nambissan, A., Astudillo, A., Yin, H., Aryani, A., 2024. Combining knowledge graphs and large language models. arXiv:2407.06564.

[18] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv:2005.11401. https://arxiv.org/abs/2005.11401.

[19] Li, X., et al., 2026. Co-MedGraphRAG: a collaborative medical QA framework. MDPI Inf. https://www.mdpi.com/2078-2489/17/3/247.

[20] Microsoft Research, 2024. GraphRAG: unlocking LLM discovery on narrative private data. Microsoft Research Blog, 2 April. https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/ (accessed 14 April 2026).

[21] Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., Tang, S., 2024. Graph retrieval-augmented generation: a survey. arXiv:2408.08921.

[22] Tonmoy, S.M., Zaman, S.M., Jain, V., Rani, A., Rawte, V., Chadha, A., Das, A., 2024. A comprehensive survey of hallucination mitigation techniques in large language models. arXiv:2401.01313.

[23] Reference Video Lecture, 2026. Graph-based retrieval-augmented generation: LightRAG implementation walkthrough. YouTube. https://youtu.be/1sTOOEwb_4I (accessed 14 April 2026).

[24] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2022. Chain-of-thought prompting elicits reasoning in large language models, in: Proc. 36th Neural Inf. Process. Syst. (NeurIPS).

[25] Yang, J., Jin, H., Tang, R., Han, X., Feng, Q., Jiang, H., Zhong, S., Yin, B., Hu, X., 2024. Harnessing the power of LLMs in practice: a survey on ChatGPT and beyond. ACM Trans. Knowl. Discov. Data 18 (6), 160:1–160:32.

[26] Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., Yang, L., Zhang, W., Cui, B., 2024. Retrieval-augmented generation for AI-generated content: a survey. arXiv:2402.19473.

A Supplementary Material

This appendix collects supporting artefacts referenced from the main body: the extraction prompt used across all LLMs, the QA evaluation questions, a representative slice of the gold reference graph, and the canonical ingest/query commands for the LightRAG-style implementation.

A.1 Extraction Prompt

The following system prompt was given to every LLM during entity/relation extraction. It was held constant across all five models in the comparison in Section 8.4 to isolate model behaviour as the sole independent variable.

> You are an expert biomedical knowledge-extraction system. From the user's text, extract all entities and the relations between them. Output a JSON object with one key `triples` whose value is a list of objects. Each triple object has the keys `subject`, `subject_type` (one of: gene, drug, disease, pathway, organisation, chemical, other), `predicate` (verb phrase in lowercase snake_case), `object`, `object_type` (same vocabulary), and `evidence_span` (verbatim quote $\leq$30 words supporting the triple). Rules: stay strictly within the supplied text, do not bring in outside knowledge; skip negations; reuse predicates across triples wherever possible; output JSON only, with no surrounding prose or markdown fences.

A.2 QA Evaluation Questions

Ten fixed questions were used to exercise each model's knowledge graph after extraction. They span single-hop lookups, multi-hop traversal, comparison, and aggregation, deliberately phrased in varied styles to penalise surface-keyword retrieval.

1. Which company developed tazemetostat?
2. What drugs treat epithelioid sarcoma and also target the EZH2 gene product?
3. What was the original tradename of trastuzumab when it was first approved?
4. Imatinib is used to treat which leukemia, and what fusion protein does it inhibit?
5. Which is more selective for EZH2 over EZH1, GSK126 or EPZ005687?
6. Which tumour-suppressor gene encodes a protein commonly called 'the guardian of the genome'?
7. Mutations in which gene predispose women to a much higher lifetime risk of breast cancer, and what type of protein does it encode?
8. List at least three EZH2 inhibitors mentioned in the corpus.
9. Pembrolizumab is a checkpoint inhibitor, which immune-checkpoint receptor does it target, and which skin cancer is it commonly used to treat?

10. Apoptosis is regulated by two main protein families, name them and say briefly what each does.

A.3 Representative Gold Triples

Sixty triples were manually annotated from the "Inhibitors" section of the EZH2 Wikipedia article and used as the reference graph for the precision/recall measurements in Table 8.3. A representative subset is shown below; the full list is maintained in the project repository as `gold/gold_triples.csv`.

Table A.1: Representative subset (15 of 60) of the manually-annotated gold reference triples.

Subject	Predicate	Object	Evidence (quoted)
DZNep	inhibits	EZH2	"it lowers EZH2 levels"
DZNep	induces	apoptosis	"induces apoptosis in breast and colon cancer cells"
EPZ005687	developed_by	Epizyme	"a company called Epizyme revealed EPZ005687"
EPZ005687	competes_with	SAM	"an S-adenosylmethionine (SAM) competitive inhibitor"
EPZ005687	selective_over	EZH1	"50-fold increase in selectivity for EZH2 compared to EZH1"
EPZ005687	binds_to	SET domain	"binding to the SET domain active site of the enzyme"
tazemetostat	developed_by	Epizyme	"Epizyme began Phase I clinical trials with . . . tazemetostat"
Tazverik	tradename_of	tazemetostat	"tazemetostat with the tradename Tazverik"
tazemetostat	treats	epithelioid sarcoma	"FDA accelerated approval for . . . epithelioid sarcoma"
tazemetostat	treats	follicular lymphoma	"approval for the treatment of patients with relapsed follicular lymphoma"
EI1	developed_by	Novartis	"developed by Novartis"
GSK126	developed_by	GlaxoSmithKline	"developed by GlaxoSmithKline"
GSK126	selective_over	EZH1	"150-fold selectivity over EZH1"
UNC1999	analogue_of	GSK126	"developed as an analogue of GSK126"
etoposide	combined_with	EZH2 inhibitor	"when combined with an EZH2 inhibitor"

A.4 Implementation Commands

The LightRAG-style implementation described in Section 7.5 exposes two command-line entry points:

Listing A.1: Ingestion: convert raw text into a Neo4j-backed knowledge graph.

```
# Parse data/data.txt, extract entities + relations,
# write to Neo4j under the configured instance
python -m run_ingest.py
```

Listing A.2: Querying: answer a natural-language question by combining vector search with graph traversal.

```
python -m run_query.py "Summarize Napoleon's Italian campaign"
```

A.5 Neo4j Visualisation

Figure A.1 shows the Neo4j Browser rendering of the knowledge graph produced by the implementation. A high-degree central node corresponding to *Napoleon Bonaparte* is surrounded by satellite entities (*Lucien Bonaparte*, *Joseph Fouché*, *Roger Ducos*, *Michael von Melas*, *Paoli*, etc.), with secondary chains connecting *The King's Swiss Guard* through *Napoleon Bonaparte* to *Toulon* and *The Insurrection*.

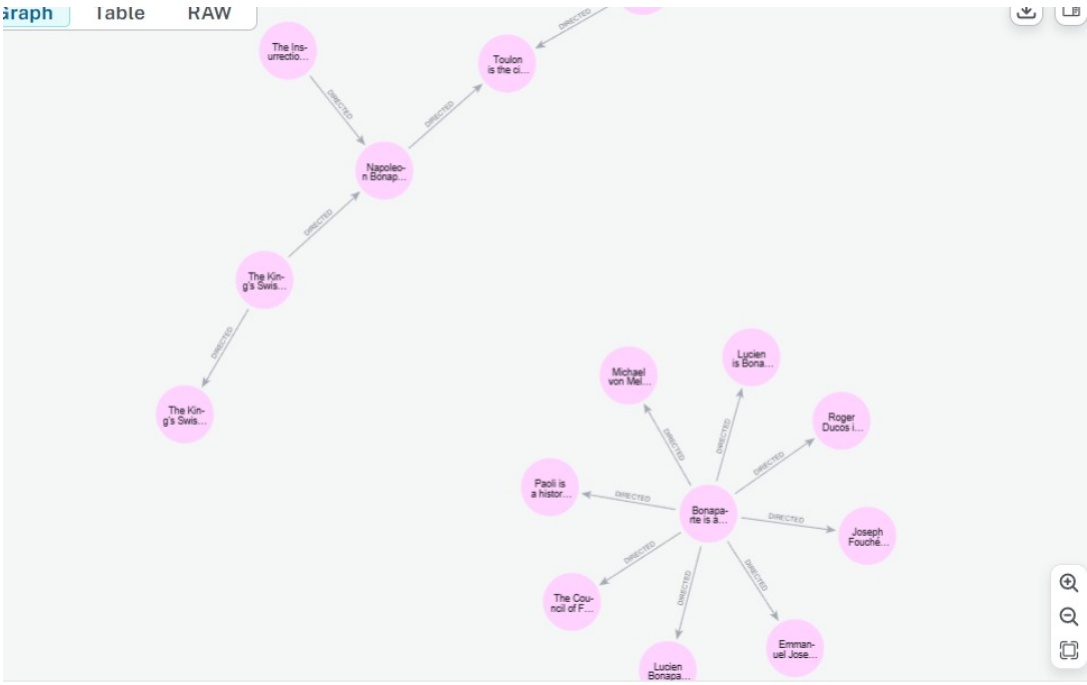

Figure A.1: Neo4j Browser view of the extracted knowledge graph. The central node corresponds to Napoleon Bonaparte and is surrounded by satellite entities linked via `DIRECTED` relationships.